```
$ lsb_release -i
Distributor ID:    LinuxMint
$ exit                              # End script session.
exit
Script done, file is typescript        # Goodbye message.
```

Now, let us look at the contents of the 'typescript' file.

```
$ cat typescript
Script started on Monday 17 February 2014 09:51:34 PM IST
$ hostname
minty
$ uname
Linux
$ lsb_release -i
Distributor ID:    LinuxMint
$ exit
exit

Script done on Monday 17 February 2014 09:51:51 PM IST
```

Yes, indeed, it stores everything from the terminal.

Command line options

Script provides many command line options to control its default behaviour. Let us see how this is done, one by one, with examples.

By default, the script command shows welcome and goodbye messages upon start-up and exit, respectively. We can suppress these messages by using the '-q' or '—quiet' option.

```
$ script -q    # Observe there is no welcome message.
$ hostname
minty
$ uname
Linux
$ exit         # Also no goodbye message.
exit
```

We can order the script command to store terminal typescript into 'file' instead of the default 'typescript' file. But here's a catch; what if 'file' already exists and has useful data. By default, script will overwrite 'file's' content. Naturally, we don't want this behaviour, so let's override this default behaviour by using the '-a' or '—append' option. Instead of overwriting, the append option appends the output to the file and retains its prior contents. The example below explains how the append option works.

Let's suppose we have a file 'imp.log' with the following contents:

```
$ cat imp.log
This file contains very important data.
```

Let us use the 'imp.log' file to store the terminal typescript with the *append* option.

```
$ script -a imp.log
Script started, file is imp.log
$ hostname
minty
$ uname
Linux
$ exit
exit
Script done, file is imp.log
```

Now observe the contents of the 'imp.log' file.

```
$ cat imp.log

This file contains very important data.
Script started on Monday 17 February 2014 10:39:58 PM IST
$ hostname
minty
$ uname
Linux
$ exit
exit

Script done on Monday 17 February 2014 10:40:07 PM IST
```

Let's congratulate ourselves! We didn't lose any important data.

Instead of providing an interactive shell, we can instruct Script to execute commands. The '-c' or '—command' option will do this job. As we are not executing the commands interactively, the log file will contain only the output of the commands and not the commands themselves. The example below will give you a better idea about the '—command' option:

```
$ script -qc "hostname; uname"
minty
Linux
```

Let us look closely at the contents of the '*typesript*' file.

```
$ cat typescript   # Observe that file contains output of the
"hostname" and "uname" command respectively.
Script started on Monday 17 February 2014 10:52:21 PM IST
minty
Linux
```

The script command ends its session by quitting the child shell gracefully. That is why it returns a zero exit code. We can use the '-e' or '—return' option to fetch the exit code of the child process. It is useful for error handling. The example below returns the correct exit code upon failure: